Codrithm Spark 2026 · Module 04

FROM CODE TO CLOUD

CI/CD & Cloud Deployment, live

MAHNOOR SAEED

DevOps Engineer · DevFusion Tech

Scan to join the live demo

sudo-mahnoor.github.io/ship-it-live

THE JOURNEY

THE LIFE OF YOUR CODE

1 **WRITE** Code on your laptop

›

2 **PUSH** Commit to Git, push to GitHub

›

3 **TEST** Automated checks run

›

4 **BUILD** Package the app

›

5 **DEPLOY** Ship it to the cloud

›

6 **MONITOR** Logs, uptime, errors

WHERE STUDENTS STOP

Step 1. The project runs on localhost, and the demo works on one laptop.

WHERE REAL PRODUCTS LIVE

Steps 2 to 6. Every change is checked, shipped and watched without anyone doing it by hand.

Writing code is one thing. Getting it into production is another.

WHY IT MATTERS

MANUAL DEPLOY VS PIPELINE

	BY HAND	WITH A PIPELINE
Steps	Copy files, restart, hope. Steps get forgotten	The same steps, in the same order, every time
Testing	Often skipped when there is a deadline	Runs on every single change
Bad code	Users find the bug first	Blocked before it reaches users
Rollback	Panic and fix it live on the server	Redeploy the last good version
History	"Who changed this?" Nobody knows	Every deploy is linked to a commit

THE WORDS

CI, CD AND CD

CONTINUOUS INTEGRATION

Push small changes often. Every push is **built and tested automatically**, so problems show up in minutes, not weeks.

CONTINUOUS DELIVERY

Every change that passes is **ready to release**. A person still presses the final "deploy" button.

CONTINUOUS DEPLOYMENT

Every change that passes **goes live automatically**. No button, no waiting.

THE GOLDEN RULE

If any step fails, nothing ships. The last good version stays live.

IN TODAY'S DEMO

We use Continuous Deployment: one git push, and your phones update.

UNDER THE HOOD

INSIDE A PIPELINE

TRIGGER

What starts it: a push, a pull request or a schedule.

JOB AND STEPS

A job is a list of steps. Each step is one command, like "run the tests".

RUNNER

A fresh cloud machine that runs the job, then disappears.

```yaml
# .github/workflows/pipeline.yml
on:
  push:
    branches: [main]   # trigger

jobs:
  test:
    runs-on: ubuntu-latest   # runner
    steps:
      - uses: actions/checkout@v4
      - run: python tests/check_site.py

  deploy:
    needs: test   # only if tests pass
    runs-on: ubuntu-latest
    steps:
      - uses: actions/deploy-pages@v4
```

WORKING IN A TEAM

PULL REQUESTS AND PROTECTED MAIN

1 **BRANCH** feature/login

2 **COMMIT** Small, clear changes

3 **PULL REQUEST** Ask to merge into main

4 **CHECKS + REVIEW** CI runs, a teammate reviews

5 **MERGE** Into main

6 **DEPLOY** Pipeline ships it

BRANCH PROTECTION RULES

- No direct pushes to main
- CI checks must pass before merge
- At least one approval from a teammate

WHY TEAMS DO THIS

- Main is always working and deployable
- Two pairs of eyes catch more bugs
- Every change has a story you can read later

SECURITY

KEEP SECRETS OUT OF CODE

NEVER

- Hardcode API keys or passwords in code
- Commit your .env file
- Paste keys in chats or screenshots

INSTEAD

- Environment variables
- GitHub Secrets or a cloud secret manager
- Different keys for dev, staging and production

IF A KEY LEAKS

- Revoke and replace it right away
- Deleting the commit is not enough. Git history keeps it
- Check the usage and the bill

```
# in the workflow file
env:
  API_KEY: ${{ secrets.API_KEY }}
```

Bots scan public GitHub repos for keys all day. A leaked cloud key can be abused within minutes.

THE CLOUD

WHERE CAN YOU DEPLOY?

STATIC SITES AND FRONTENDS

GitHub Pages, Vercel, Netlify

Portfolios, landing pages, React and Next.js apps. Free tiers to start.

FULL-STACK APPS AND APIS

Render, Railway, Fly.io

Backends, databases, bots. Push code and the platform runs it.

FULL CONTROL AT SCALE

AWS, Azure, Google Cloud

Servers, networks, Kubernetes. What most companies run on.

DOCKER

Package your app with everything it needs. It runs the same on your laptop, in CI and in the cloud.

INFRASTRUCTURE AS CODE

Tools like Terraform describe servers in code. No more clicking around a console and forgetting what you did.

WHAT TO LEARN NEXT

YOUR DEVOPS ROADMAP

STEP 1
GIT AND GITHUB
Branches, pull requests, clean commits

STEP 2
GITHUB ACTIONS
Your first CI/CD pipeline

STEP 3
LINUX BASICS
Terminal, SSH, processes, networking

STEP 4
DOCKER
Containers and images

STEP 5
ONE CLOUD
AWS, Azure or GCP fundamentals

STEP 6
TERRAFORM
Infrastructure as Code

STEP 7
MONITORING
Logs, metrics and alerts

START TODAY
SHIP ONE PROJECT
End to end, with a pipeline. Put the live link on your CV.

TAKE THIS HOME

4 THINGS TO REMEMBER

1. Manual deploys break things.
2. A pipeline tests, builds and deploys for you.
3. Bad code gets stopped before users see it.
4. Secrets never go in code.

DEPLOYING IS EASY. SHIPPING SAFELY IS THE SKILL.

THANK YOU

QUESTIONS?

MAHNOOR SAEED

DevOps Engineer · DevFusion Tech

Fork the demo repo, build your own pipeline, and send me the live link.

Connect on LinkedIn

Fork the demo repo

PLAN B

THE WI-FI DIDN'T PASS ITS TESTS